OOP in Python

Input Code and Output

1. Default Constructor

Input Code

```
class ATM:
    def __init__(self):
        self.pin = None
        self.balance = 0
        print("ATM system initialized!")
        self.menu()

    def menu(self):
        print("Hello! How can I help you?")
        print("1. Set PIN")
        print("2. Change PIN")
        print("3. Check Balance")
        print("4. Withdraw Cash")

atm1 = ATM()
```

Output

```
ATM system initialized!
Hello! How can I help you?
1. Set PIN
2. Change PIN
3. Check Balance
4. Withdraw Cash
```

2. Initializer / Instance Variables

Input Code

```
class Addition:
    def __init__(self):
        self.num1 = 1000
        self.num2 = 2000
        self.num3 = 3000

    def result(self):
        x = 10
        self.num = self.num1 + self.num2 + self.num3 + x
        print("Output", self.num)

Sum = Addition()
Sum.result()
```

Output

```
Output 6010
```

3. Class Variable

Input Code

```
class Demo:
    x = 100

a = Demo()
b = Demo()

print(a.x)
print(b.x)
```

Output

```
100
100
```

4. Parameterized Constructor

Input Code

```
class Student:
    def __init__(self, name, ids, college):
        print("This is a parameterized constructor in python")
        self.name = name
        self.ids = ids
        self.college = college
```

```
    def Display_Details(self):
        print("Student Details")
        print("Student Name", self.name)
        print("Student ids", self.ids)
        print("Student college", self.college)

student = Student("JOHN", 2023, "MIT")
student1 = Student("rahul", 2023, "MIT")

student.Display_Details()
student1.Display_Details()
```

Output

```
This is a parameterized constructor in python
This is a parameterized constructor in python
Student Details
Student Name JOHN
Student ids 2023
Student college MIT
Student Details
Student Name rahul
Student ids 2023
Student college MIT
```

5. Student Details

Input Code

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

s1 = Student("Rahul", 85)
s1.display()
```

Output

```
Name: Rahul
Marks: 85
```

6. Default Arguments in Constructor

Input Code

```
class Employee:
    def __init__(self, name="Not Assigned", salary=0):
        self.name = name
        self.salary = salary

    def display(self):
        print(self.name, self.salary)

e1 = Employee()
e2 = Employee("Amit", 40000)
e1.display()
e2.display()
```

Output

```
Not Assigned 0
Amit 40000
```

7. Bank Account

Input Code

```
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance = self.balance - amount

    def display(self):
        print("Account Holder:", self.name)
        print("Balance:", self.balance)
```

```
acc = BankAccount("Sneha", 10000)
acc.deposit(2000)
acc.withdraw(1500)
acc.display()
```

Output

```
Account Holder: Sneha
Balance: 10500
```

8. Single Inheritance

Input Code

```
class Parent:
    def func1(self):
        print("This function is in parent class.")

class Child(Parent):
    def func2(self):
        print("This function is in child class.")

obj = Child()
obj.func1()
obj.func2()
```

Output

```
This function is in parent class.
This function is in child class.
```

9. Multiple Inheritance

Input Code

```
class Mother:
    mothername = ""

    def mother(self):
        print(self.mothername)

class Father:
    fathername = ""

    def father(self):
        print(self.fathername)

class Son(Mother, Father):
    def parents(self):
        print("Father :", self.fathername)
        print("Mother :", self.mothername)

s1 = Son()
s1.fathername = "RAM"
s1.mothername = "SITA"
s1.parents()
```

Output

```
Father : RAM
Mother : SITA
```

10. Method Resolution in Multiple Inheritance

Input Code

```
class Class1:
    def m(self):
        print("In Class1")

class Class2(Class1):
    def m(self):
        print("In Class2")

class Class3(Class1):
    def m(self):
        print("In Class3")

class Class4(Class2, Class3):
    pass

obj = Class4()
obj.m()
```

Output

```
In Class2
```

11. Method Resolution in Multiple Inheritance (Second Example)

Input Code

```
class Class1:
    def m(self):
        print("In Class1")

class Class2(Class1):
    pass

class Class3(Class1):
    def m(self):
        print("In Class3")

class Class4(Class2, Class3):
    pass

obj = Class4()
obj.m()
```

Output

```
In Class3
```

12. Calling Parent Class Methods

Input Code

```
class Class1:
    def m(self):
        print("In Class1")

class Class2(Class1):
    def m(self):
        print("In Class2")
        Class1.m(self)

class Class3(Class1):
    def m(self):
        print("In Class3")
        Class1.m(self)

class Class4(Class2, Class3):
    def m(self):
        print("In Class4")
        Class2.m(self)
        Class3.m(self)

obj = Class4()
obj.m()
```

Output

```
In Class4
In Class2
In Class1
In Class3
In Class1
```

13. Super Function

Input Code

```
class Class1:
    def m(self):
        print("In Class1")

class Class2(Class1):
    def m(self):
        print("In Class2")
        super().m()

class Class3(Class1):
    def m(self):
        print("In Class3")
        super().m()

class Class4(Class2, Class3):
    def m(self):
        print("In Class4")
        super().m()

obj = Class4()
obj.m()
```

Output

```
In Class4
In Class2
In Class3
In Class1
```

14. Multilevel Inheritance

Input Code

```
class Grandfather:
    def __init__(self, grandfathername):
        self.grandfathername = grandfathername

class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername
        Grandfather.__init__(self, grandfathername)

class Son(Father):
    def __init__(self, sonname, fathername, grandfathername):
        self.sonname = sonname
        Father.__init__(self, fathername, grandfathername)

    def print_name(self):
        print('Grandfather name :', self.grandfathername)
        print('Father name :', self.fathername)
        print('Son name :', self.sonname)

s1 = Son('ABC', 'MUKESH', 'DHIRUBHAI')
print(s1.grandfathername)
s1.print_name()
```

Output

```
DHIRUBHAI
Grandfather name : DHIRUBHAI
Father name : MUKESH
Son name : ABC
```

15. Hierarchical Inheritance

Input Code

```
class Parent:
    def func1(self):
        print("This function is in parent class.")

class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")

class Child2(Parent):
    def func3(self):
        print("This function is in child 2.")

object1 = Child1()
object2 = Child2()

object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

Output

```
This function is in parent class.
This function is in child 1.
This function is in parent class.
This function is in child 2.
```

16. Shape Area Using Inheritance

Input Code

```
class Shape:
    def __init__(self, color):
        self.color = color

    def area(self):
        pass

class Circle(Shape):
    def __init__(self, color, radius):
        super().__init__(color)
```

```
        self.radius = radius

    def area(self):
        return 3.14 * self.radius**2

class Square(Shape):
    def __init__(self, color, side_length):
        super().__init__(color)
        self.side_length = side_length

    def area(self):
        return self.side_length**2

circle = Circle("Red", 5)
square = Square("Blue", 4)
print(circle.area())
print(square.area())
```

Output

```
78.5
16
```

17. Hybrid Inheritance

Input Code

```
class School:
    def func1(self):
        print("This function is in school.")

class Student1(School):
    def func2(self):
        print("This function is in student 1.")

class Student2(School):
    def func3(self):
        print("This function is in student 2.")

class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")

obj = Student3()
obj.func1()
obj.func2()
```

Output

```
This function is in school.
This function is in student 1.
```